Dukaan

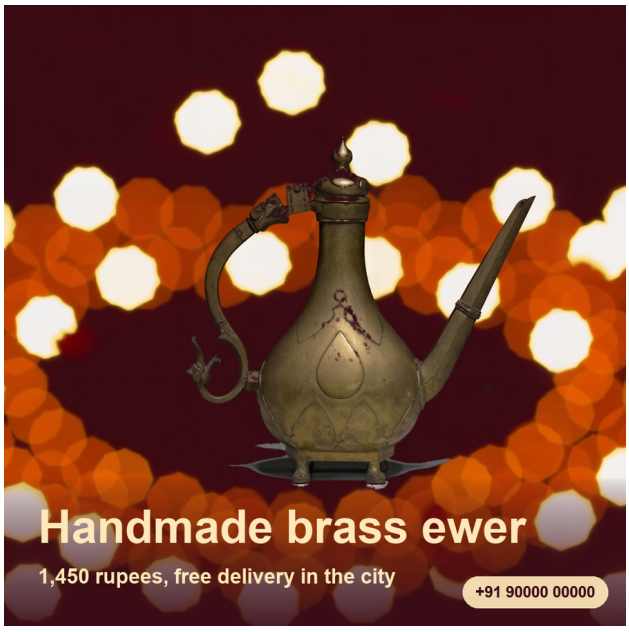
One product photo in, a shop's worth of creatives out. Generated on AMD Radeon.

Track 1, Development of Multimodal Content Creation Tools himanshu748 | Radeon PRO gfx1100, 48 GB | ROCm 7.2.4 | LTX-2.3 22B

The problem

A shopkeeper photographs stock in ten seconds. Turning that into something postable takes an afternoon in a design tool, per product, every time the catalogue turns over. Tools that close the gap usually generate the product too, which is the wrong trade: a seller cannot post a picture of a bangle that is not the bangle they will ship.

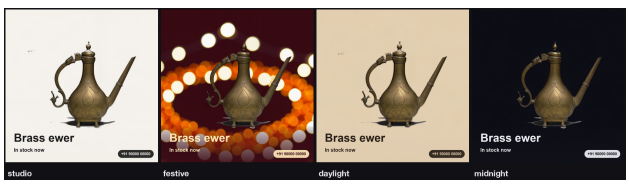
What Dukaan does



One photo becomes a square for the feed, a 9:16 for stories, a wide banner for a shop header, and a short clip with generated ambient audio. The product is screened against the input plate and shown for review. The price and phone number are composited with real type, never drawn by a model.

The GPU runs once per pack, not once per format. Three formats are three different moments of the same generated clip.

One photo, four looks, with the input plate available for comparison:



The finding: the instance cannot run its own template

LTX-2.3 checkpoint	43 GB
Gemma 3 12B text encoder	23 GB
Total to load	**66 GB**
Container cap	**55 GB**

ComfyUI holds every model a graph touches for the life of the process. Loading both trips the cap and **the platform restarts the container mid-prompt**: JupyterLab returns with zero kernels, the port stops answering, and it presents as a network fault.

The fix: two processes that never overlap

phase	peak container RAM	wall
encode, text encoder only	35.4 GB	33 s
sample, checkpoint only	51.2 GB	55 s
stock template, one process	*trips 55 GB, restarts*	*n/a*

Peak becomes `max(43, 23)`, not `43 + 23`.

Plus: bf16 conditioning (it is read back while the checkpoint is resident), VRAM eviction before the VAE decode (the decode died with 2.13 GB free out of 47.98), and `torch.inference_mode()` around the phase (the LTX VAE updates the sampler's output in place).

Measured

setting	output	GPU	peak RAM
768x768, 49 frames	768x768	70.7 s	49.9 GB
768x768, refined	**1536x1536**	176.1 s	49.9 GB
3 products, one batch	768x768	**134.3 s**	49.9 GB

Every run holds under the 55 GB cap. `dukaan bench` reproduces the table.

Transport was a bottleneck too: 49 frames as 49 base64 requests took 3 m 34 s and reset the tunnel twice. One tar brought it to 2 m 19 s.

Batching is the lever that removes work rather than trading it

Loading the checkpoint costs 17 s and the text encoder 9 s, whatever you then generate. A shop packing fifty items one at a time pays that fifty times.

3 products, one batch	**134.3 s**
3 products, separately	212.1 s
per-product, across the batch	36.3 s, then 28.0 s, then 26.6 s

The 13.8 s load is paid once, and per-product time then falls as the GPU warms, for identical work. A 37% saving on three; it grows with the catalogue.

Three CPU decisions that keep GPU credits for the GPU

The cutout fits the background, it does not sample it. A quadratic surface per channel on the border ring, refit once with the worst residuals dropped. A plane left an elliptical pool of backdrop exactly where the product sits.

Stills are picked for spread, reference consistency and sharpness. The reference signal gets most of the score inside each contiguous window, with sharpness preventing a soft frame from winning. Frame 0 is never eligible: it is the plate the model was handed.

Reshaping mirrors, it does not crop or stretch. Cover-cropping a square to 9:16 cuts the product; clamping the edge row drew the bokeh into horizontal streaks.

Automated tests, no GPU required. They cover MP4 export, safe output paths, phone EXIF orientation, reference-aware selection and catalogue audit records. Without an instance configured, the whole pipeline runs on CPU and writes real files, so the tool can be inspected before any spend. Apache-2.0. Demo photographs are CC0.